By convention, Compose names the network <appname>_default, where appname is inferred from the containing directory.

Next it builds the image for our web service:

```
Building web
Step 1/8 : FROM ruby:2.6
 ---> f28a9e1d0449
Step 2/8 : LABEL maintainer="rob@DockerForRailsDevelopers.com"
 ---> Using cache
 ---> d69ea7d90f89
Step 3/8 : RUN apt-get update -yqq && apt-get install -yqq --no-install-
recommends   nodejs
 ---> Using cache
 ---> 463750079bef
«...»
Step 8/8 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Running in b11e989011fc
Removing intermediate container b11e989011fc
 ---> a18b3079c84b
Successfully built a18b3079c84b
```

Compose gives the image a name and version by tagging it:

```
Successfully tagged myapp_web:latest
```

using the convention <appname>_<service_name>:latest, again inferring the appname from the enclosing directory. In our case, this becomes myapp_web:latest.

You can verify that the myapp_web image was created by running the following in a separate terminal window:

```
$ docker images
```

You should see it listed in one of the lines of the output:

```
REPOSITORY      TAG         IMAGE ID          CREATED                 SIZE
myapp_web       latest      a18b3079c84b      About a minute ago      1.01GB
...
```

Compose will only build images if they don't exist, which will either be because it's the first time you've run docker-compose up or because you've deleted them. This is an important point: you are responsible for rebuilding your images as needed (see Rebuilding Our Images, on page 55); in fact, Compose reminds us of this in the output:

```
WARNING: Image for service web was built because it did not already exist.
To rebuild this image you must use `docker-compose build` or `docker-compose
up --build`.
```